

TABLE OF CONTENT

CHAPTER	CONTENTS	PAGE NO.
1	INTRODUCTION	2
1.1	MOTIVATION	2-3
1.2	LITERATURE SURVERY	3-4
1.3	PROBLEM FORMULATION	5
1.4	PROJECT OBJECTIVES	5-6
2	HARDWARE AND SOFTWARE REQUIRMENTS	7
2.1	HARDWARE REQUIRMENTS	7
2.2	SOFTWARE REQUIRMENTS	7
2.3	HEALTH MONITORING SYSTEM SPECIFICATIONS	8-10
3	METHODOLOGY	11
3.1	FLOWCHART	11
3.2	BLOCK DIAGRAM	12
3.3	COMPONENTS DESCRIPTION	13
4	RESULT AND DISCUSSIONS	14
4.1	HARDWARE PROTOTYPE	14-16
4.2	ADVANTAGES	17
4.3	DISADVANTAGES	17
4.4	APPLICATIONS	18
4.4	FUTURE SCOPE OF WORK	18-19
	REFERENCE	20-21
	APPENDIX	22-32

CHAPTER-1

INTRODUCTION

The paradigm of modern healthcare is shifting toward proactive and preventative care. In an era where personal well-being is paramount, the ability to continuously and effortlessly monitor one's health has become a crucial aspect of modern life. Traditional vital sign assessments, typically performed sporadically in clinical settings, provide only temporal snapshots and fail to capture crucial health dynamics. Relying on patients to show symptoms before seeking medical attention is inadequate; a continuous monitoring system is required to provide an early warning for potential underlying issues.

The COVID-19 pandemic further accelerated the demand for remote health monitoring, highlighting the risks of in-person check-ups and the necessity of tracking patient health from a distance. The objective of this project is to address this need by developing a system for continuous, non-invasive vital signs acquisition.

- The traditional healthcare model often requires travel for routine check-ups. A portable system allows individuals to monitor their vital signs from home, which is particularly beneficial for the elderly, patients with chronic conditions, and individuals in remote areas. The system, built with low-cost components, is an affordable alternative to professional medical-grade devices.
- IoT Integration: Beyond local use, this project aims to support Remote Healthcare Applications. Technologies like Wireless Communication (Bluetooth, Wi-Fi, Zigbee) and the Internet of Things (IoT) framework allow the system to transmit data to a central hub or cloud server, enabling continuous monitoring and data analytics. This enables healthcare professionals to track a patient's health status remotely and allows for Timely Alerting when vital signs deviate from a safe range.

1.1. MOTIVATION:

The development of this low-cost, portable health monitoring system is primarily motivated by the critical need to overcome the limitations of the traditional healthcare model and leverage modern technological advancements for improved patient outcomes.

1.1.1 Bridging the Gaps in Traditional Healthcare:

The primary motivation is to solve systemic challenges within the current healthcare framework, which is often reactive, costly, and inconvenient.

- **Ensuring Accessibility and Convenience:** The traditional model requires patients to travel for routine check-ups, which is time-consuming and limited to specific clinical settings. A portable system allows individuals to monitor their vital signs from the comfort of their home. This is particularly beneficial for elderly individuals, patients with chronic conditions, and individuals in remote areas with limited access to healthcare facilities.

- **Achieving Cost Reduction:** Remote monitoring can significantly reduce overall healthcare costs. It decreases the frequency of in-person doctor visits, reduces hospital readmissions, and prevents the need for expensive emergency care by catching health issues early. The system itself is built with low-cost components, making it an affordable alternative to professional medical-grade devices.

1.1.2. Enabling Data-Driven and Empowered Health Management:

The project is motivated by the potential for technology to empower individuals and enhance clinical analysis:

- **Data-Driven Insights:** A continuous stream of data on pulse and temperature provides a more comprehensive picture of a person's health than a single point-in-time measurement. This data can be stored, analyzed, and used by healthcare providers to identify trends, evaluate treatment effectiveness, and create personalized care plans.
- **Personal Health Management:** Beyond clinical use, the project empowers individuals to take charge of their own health. Real-time feedback promotes a greater awareness of the body's state and encourages healthy habits. An athlete, for instance, can use it to monitor their heart rate during a workout.

1.2. LITERATURE SURVEY:

This review summarizes the core concepts and findings of various research initiatives focused on developing remote patient health monitoring systems utilizing the ESP32 microcontroller.

Nowadays, in most countries the death is increasingly because health problem day by a day. Each year the rate of death is 55.3 million of people, this rate due to a delay in giving proper treatment or lack of care due to lack of medical staff or resource especially in rural hospital. Hence, there is a need for some techniques to overcome this problem. This paper proposed a design of an intelligent healthcare monitoring system based on IOT [1 & 2].

Internet of thing (IOT) is a concept of network that is connected several devices (portable, wearable, house – held device etc...), network internally, also combine software with hardware in network, and connected this with other network. IOT environment allow supervising control or transferring remotely from one network to another one in that infrastructure. In the healthcare field, IOT helps to real-time monitoring system with more flexibility and important vital function of the people in any time and place. Therefore, it can transfer the data acquired remotely in real-time, which ensure real-time monitoring to patient status [1, 3 & 4].

Data Acquisition With ESP32: ESP32 is a microcontroller designed to handle with low power and low cost system, and it has the possibility and capabilities of Arduino, as it can be dealt with programmatically through the Arduino IDE and Micro-python environment. Esp32 is a high performance 32-bit LX6 microprocessor, ultra-low power co-processor 3.3VDC. It has 6-chanel 12-bit input ADC pins, 2-chanel output DAC pins, 3 URTs, 2-SPIs, 3-I2C, flash memory 4MB, in addition to 520KB SRAM [5 & 6].

Across the globe, COVID-19 had far-reaching impacts that included healthcare facilities, public health, as well as all forms of transport. Hospitals were experiencing staffing shortages at the same time as patients were experiencing healthcare issues. Consequently, even in developing countries without full access to technology, remote health monitoring became necessary. To facilitate remote health monitoring, an IoT server has been configured on an ESP32 chip as part of this study. The microcontroller was also connected to a Max 30100 sensor, a DHT11 sensor, and a global positioning system GPS module. As a result of this, the user is able to measure the heart rate (HR), human body temperature, ambient temperature and humidity, as well as the location of the user. Through the internet protocol, the important vital signs can be displayed in real time on the dashboard using a private communication network [7].

With the improvement of people's quality of life, the environmental requirements of laboratories and agricultural production, there are higher requirements for the temperature of the environment. The system realizes the real-time detection of the temperature of the detection place through esp32 development board and various sensors, obtains data, and can feed back in real time and remotely through the cloud server, and uses the cloud server to make statistics on the temperature and humidity and draw charts [8].

The proposed system is able to measure the heart rate/pulse (BPM), as well as the blood oxygen level (SpO₂), using the MAX30100/102 pulse oximeter sensor. Some DS18B20 temperature sensors are used to measure the average body temperature. The temperature and humidity inside a room are detected with a DHT11 Humidity and Temperature sensor. The MQ-135 sensor is needed for monitoring the noxious gas levels that can affect the air quality inside a room [8].

Data is sent to/from the DS18B20 via a 1-wire interface, so only one wire (ground) needs to be connected from a central microprocessor to the DS18B20. It provides power for reading, writing and temperature conversions. The values can be derived from the data bus itself, without the need for an external power supply. Because each DS18B20 contains a unique serial number on silicon, multiple DS18B20s can exist on the same 1-wire bus. This allows temperature sensors to be placed in various places [9].

The MQ-135 air quality sensor is used to measure the air quality inside the room where the patient is placed [10]. The air quality is constantly monitored with the developed system, so that the amount of toxic gas, smoke or alcohol cannot become dangerous for the patient. In case the gas level is exceeded, an alarm is triggered. A thermistor is a sensitive thermal resistor that uses resistance values to determine temperature. It is comprised of a semiconductor, and when subjected to temperature changes, its physical resistance changes. The DHT11 sensor is used for measuring ambient temperature and humidity, factors that decide whether the environment is conducive to certain bacteria or viruses [11].

The MAX30100 pulse sensor is a combined heart rate and blood oxygen sensor. The sensor is meant to monitor blood oxygen levels and the pulse using photoplethysmography, a non-invasive technique. The sensor is an integrated photodetector and LED emitter. Infrared (IR) and red light are emitted by the LED emitter, and the photodetector gauges how much light is reflected off the user's skin. For the purpose of detecting variations in blood vessel volume during the pulse, photoplethysmography is used. The sensor detects fluctuations in the absorption of infrared and red light as the pulsing blood moves through the arteries [12 & 13].

1.3. PROBLEM FORMULATION:

1.3.1. The Core Problem Statement:

The fundamental problem this project addresses is to design and develop a low-cost, portable, and user-friendly health monitoring system that can continuously and accurately measure pulse and body temperature, thereby overcoming the limitations of traditional healthcare by providing an accessible, proactive, and data-driven solution for at-home vital sign monitoring.

1.3.2. Specific Challenges to be Overcome:

The system must successfully resolve the following challenges inherent in current healthcare models and system design:

- **Accessibility and Convenience Barriers:** Overcome the reliance on clinical settings by allowing individuals, especially the elderly and chronic patients, to monitor vital signs from the comfort of their home.
- **Lack of Proactive Monitoring:** Transition from reactive treatment (waiting for symptoms) to proactive monitoring that provides an early warning system for potential anomalies like sudden spikes in temperature or irregular pulse rates.
- **Data Fragmentation and Silos:** Address the issue of health data being siloed and difficult to analyze over time by providing a continuous, unified data stream.
- **Cost of Devices:** Develop a system using affordable components (e.g., Arduino, basic sensors) as a cost-effective alternative to expensive, professional medical-grade devices.

1.4. PROJECT OBJECTIVES:

The objectives are the specific, measurable goals the project aims to achieve to overcome the limitations of traditional healthcare and provide an accessible, proactive, and data-driven solution.

1.4.1. Data Acquisition and Measurement Accuracy:

- **Real-Time Data Collection:** To accurately and continuously measure the defined vital signs of the user in real-time. The system must continuously and accurately measure pulse and body temperature.
- **Hardware Integration:** To successfully select, integrate, and deploy the chosen sensors (Pulse Oximeter/Sensor, DS18B20) with the ESP32 microcontroller.
- **Signal Processing:** To develop robust firmware (Arduino program) capable of processing raw sensor data (including signal conditioning and filtering) to calculate and verify accurate vital sign outputs.

1.4.2. User Interface and Alerting:

- **Local Display:** To display the measured vital signs (pulse and temperature) directly to a small, on-device screen (e.g., OLED or LCD).
- **Timely Alerting:** To automatically detect abnormal health conditions (when a vital sign deviates from a safe range) and send immediate alerts to the patient, caregivers, and/or medical staff.

1.4.3. Connectivity and Remote Monitoring (IoT Integration):

- **Implement IoT Connection:** To establish a secure data transmission network using Wi-Fi setup and cloud integration with the ESP32.
- **Enable Remote Monitoring:** To allow healthcare professionals to track a patient's health status remotely, enabling intervention regardless of the patient's location.
- **Data Storage:** To develop a mechanism for storing the continuous stream of data, potentially using local storage (SD/EEPROM) or a central cloud platform/server.

1.4.4. Portability and Cost-Effectiveness:

- **Low-Cost Design:** To build the system using readily available and affordable components, providing an affordable alternative to professional medical-grade devices.

CHAPTER-2

HARDWARE AND SOFTWARE REQUIREMENTS

This chapter contains the detailed information about the hardware and software requirements for the project and its hardware and software integrations, architecture, hardware interfacing and components. It gives complete specification about health monitoring system.

2.1. HARDWARE REQUIREMENTS:

The Hardware/Device scope focuses on the physical realization of the monitoring unit, ensuring reliable data acquisition while maintaining the project's core requirement of low-cost portability.

- **Sensor Selection and Deployment:** This involves the selection, integration, and deployment of sensors—which can be wearable, embedded, or stationary—to reliably collect patient data. The chosen sensors include the DS18B20 for temperature and a Pulse Oximeter Sensor or basic pulse sensor for heart rate, which uses the principle of photoplethysmography (PPG).
- **Local Data Output:** A crucial constraint is that the system must display the measured vital signs directly to a small, on-device screen (e.g., LCD or OLED).
- **Portability and Power:** To achieve portability, the hardware scope includes the power management components, specifically a battery.

2.2. SOFTWARE REQUIREMENTS:

2.2.1. Arduino IDE Software:

- **Programming Language:** Primarily C/C++ (used within the Arduino framework).
- **Sensor Libraries:** Specific libraries required to interface and communicate with the chosen digital sensors (DS18B20, etc.).
- **Display Libraries:** Libraries required to drive and update the data on the selected device display.
- **Signal Processing Logic:** Algorithms and code for filtering noise and executing peak detection on the raw PPG signal to accurately calculate the BPM.
- **Alerting Logic:** Code to continuously monitor measured values against defined safe ranges and trigger the alert module.

2.2.2. IoT and Data Transmission:

- **Wireless Communication Implementation:** Firmware libraries and code necessary to implement the Wi-Fi setup on the ESP32.

- **Cloud Platform/Server:** A central cloud platform or server is required for data storage and processing, enabling the core objective of remote monitoring and real-time data monitoring.

2.3. HEALTH MONITORING SYSTEM SPECIFICATIONS:

This section deals with the technical architecture, sensor integration, and data flow of our advanced Health Monitoring System. Below is the full description organized in the correct, logical sequence, from hardware connection to data processing and alerting.

2.3.1. System Architecture and Sensor Integration:

The system integrates four distinct sensors onto the **ESP32 microcontroller** to measure both human health and environmental conditions, leveraging the ESP32's versatility in handling multiple communication protocols.

Sensor	Parameter Measured	Communication Protocol	ESP32 Interfacing
MAX30100	Heart Rate (BPM)	I ² C (SDA, SCL)	GPIO 21 (SDA), GPIO 22 (SCL)
DS18B20	Body Temperature	One-Wire	Digital GPIO pin (e.g., GPIO 4)
DHT22	Ambient Temperature & Humidity	Single Wire (Digital)	Digital GPIO pin (e.g., GPIO 5)
MQ-135	Air Quality (Gas Concentration)	Analog Output	ADC pin (e.g., GPIO 36)

2.3.2. Hardware interfacing with ESP32:

The ESP32 is specifically configured to manage these diverse protocols:

- **MAX30100/102:** The SDA and SCL pins connect to the ESP32's default I²C pins (GPIO 21 and GPIO 22), allowing for fast, standard two-wire communication necessary for continuous pulse data transfer.
- **DS18B20 (One-Wire):** The data line is connected to a single digital GPIO pin (e.g., GPIO 4) and requires a 4.7 k Ω pull-up resistor connected between 3v3 and d5 pin for reliable communication.

- **DHT22 (Digital):** The data line is connected to a single digital GPIO pin (e.g., GPIO 5) and typically requires a 10 k Ω pull-up resistor on the data line.
- **MQ-135 (Analog):** The Analog Output (A0) pin is connected to an Analog-to-Digital Converter (ADC) pin (e.g., GPIO 34). Crucially, the non-linear characteristic of the ESP32's ADC and its range necessitate software calibration for accurate gas concentration readings.

2.3.3. Software and data processing flow:

The software environment manages sensor communication, complex signal processing, and external data transmission.

Required Libraries:

The following libraries are essential for handling the various protocols and calculations:

- **Connectivity:** WiFi.h for network access.
- **I²C:** Wire.h for MAX30100 communication.
- **Heart Rate:** MAX30105.h to process raw Photoplethysmography (PPG) data.
- **Temperature:** OneWire.h and DallasTemperature.h for the DS18B20.
- **Ambient:** DHT.h for the DHT22.

2.3.4. Data Processing and Signal Analysis:

- **Heart Rate Processing:** The dedicated library processes the raw PPG data from the MAX30100 by detecting the periodic changes in reflected light. It then calculates the Heart Rate in Beats Per Minute (BPM) based on the measured time interval between consecutive beats (pulse detection). The overall process can be broken down into three main stages: Light Emission and Detection, Signal Processing (by the MAX30100), and Data Communication and Calculation (by the ESP32).

Light Emission: The MAX30100 module contains two built-in LEDs: a Red LED and an Infrared (IR) LED. For heart rate (HR) measurement alone, the IR LED is typically used.

Light Absorption: When the fingertip is placed on the sensor, the LED light shines into the skin. As blood flows through the capillaries, it absorbs some of this light.

Blood Volume Change: With each heartbeat, the volume of blood passing through the finger capillaries momentarily increases, which in turn causes more light to be absorbed and less light to be reflected back to the sensor. Between heartbeats, the blood volume decreases, so less light is absorbed, and more light reflects back.

Light Detection: A photodetector on the MAX30100 module measures the amount of light that reflects back from the fingertip. This detected light intensity varies rhythmically with your pulse and this data is given to the ESP32 for processing.

- **Temperature and Humidity Processing:** Readings from the DS18B20 and DHT22 are converted into percentage of humidity and temperature using their respective libraries.
- **Air Quality Processing:** The analog reading from the MQ-135 is converted by the ESP32's ADC and must be passed through a critical calibration routine in the code to translate the raw voltage into a meaningful air concentration indicator.

2.3.5. Data Transmission and Alert System:

The final stages focus on realizing the project's remote monitoring objective through wireless communication and intelligent alerting.

Data Transmission Methods: The ESP32 utilizes its integrated Wi-Fi module to transmit all collected and processed data:

- **Local Web Server:** The ESP32 can host a simple web page using WebServer.h. Users connected to the same Wi-Fi network can view real-time data by accessing the ESP32's IP address in a web browser.

2.3.6. Alert System Functionality:

The code implements a critical alert system that continuously checks measured readings against pre-defined thresholds:

- **Health Alerts:** If the Heart Rate is too low (Bradycardia) or too high (Tachycardia), or if Body Temperature exceeds a fever threshold, an alert is triggered.
- **Environmental Alerts:** If the Air Quality (MQ-135 concentration) exceeds a safety limit, an environmental alert is triggered.

CHAPTER – 3**METHODOLOGY**

This Chapter mainly describes the project methodology and its flowchart, how the system integrated will work in an order and the flow of its working is mention in this chapter.

3.1. FLOW CHART:

The collected data is processed using the ESP32 microcontroller and can be monitored in real-time through an IoT platform or display device.

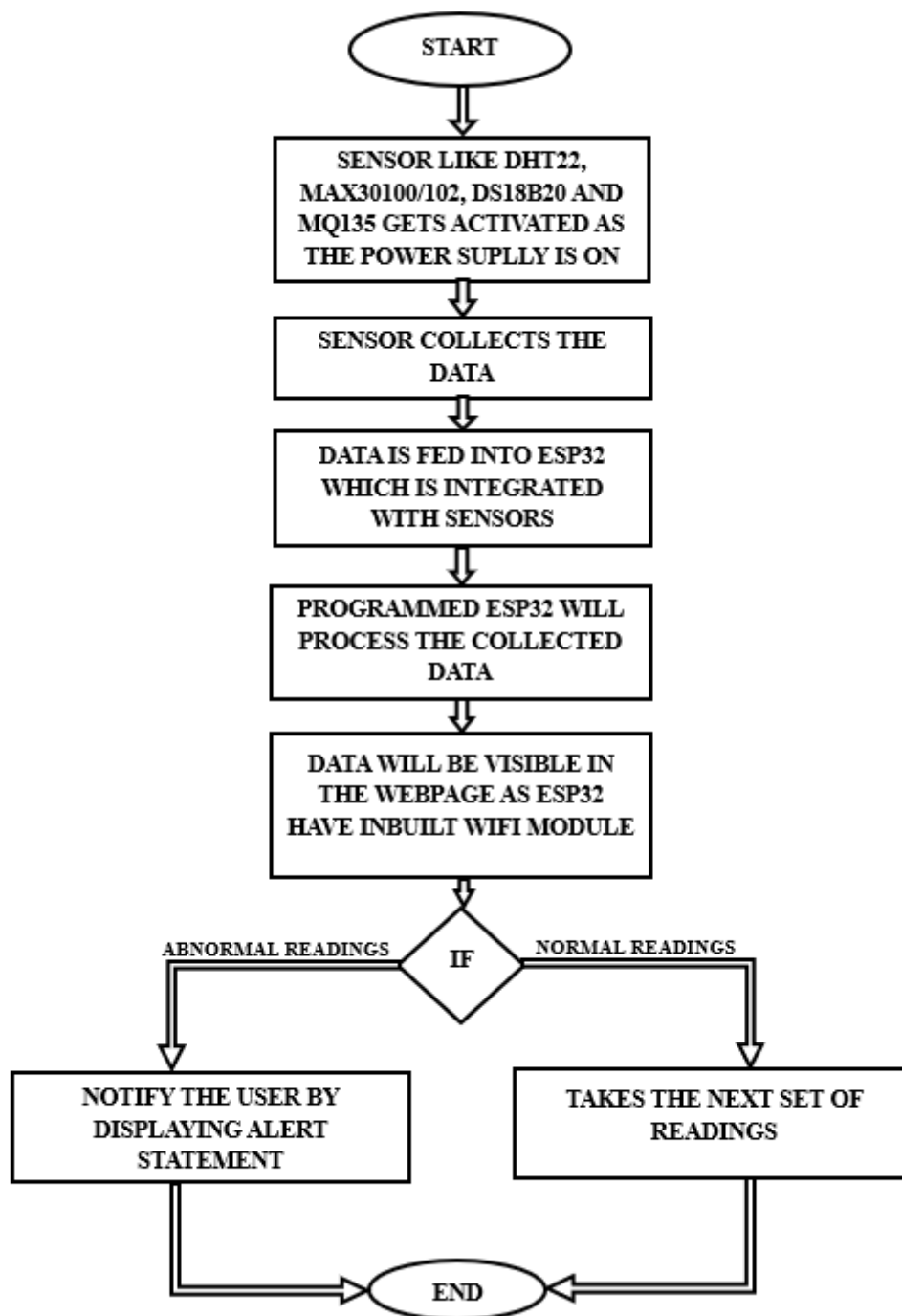


Fig.1. Flow chart describing the working process

3.2. BLOCK DIAGRAM:

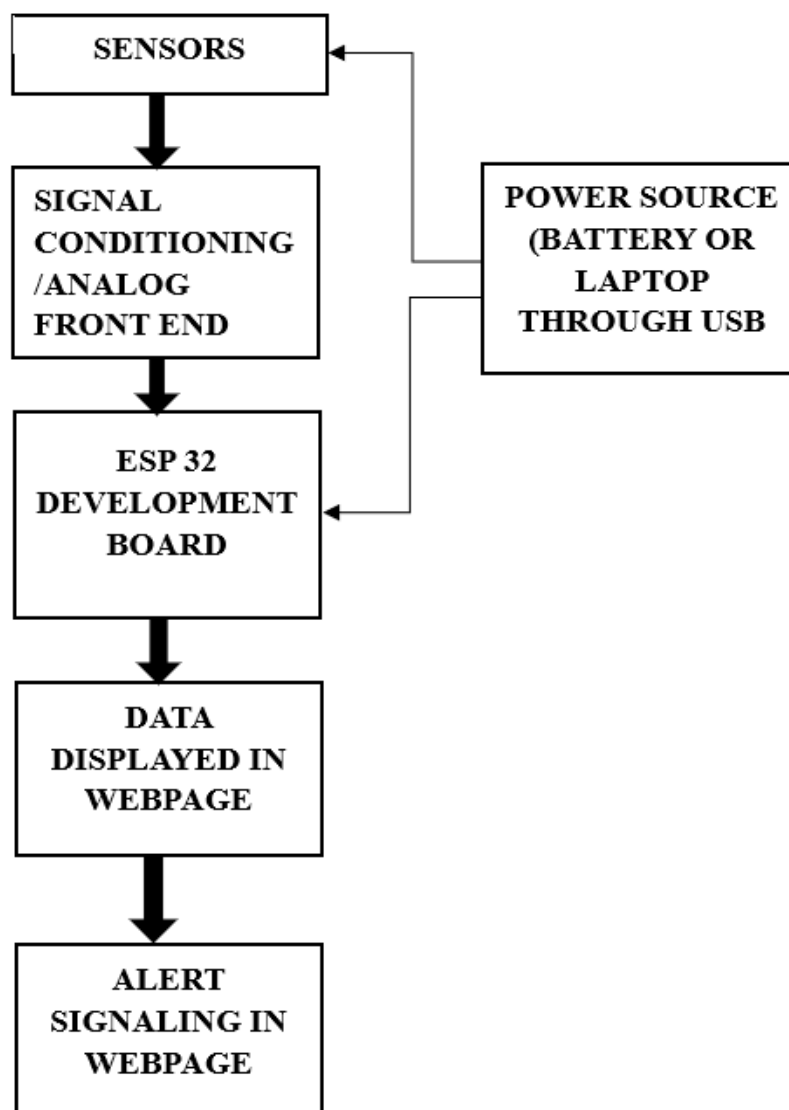


Fig.2. Block diagram of the system

This the flow diagram of our system design which shows the detailed design flow . Mainly the sensors and microcontroller (esp32) is powered by the power source, either a battery or by using a USB cable with laptop, preferred to use laptop for better power delivery. Then the sensors that is integrated with the esp32 with first convert all the analog signals into digital during signal condition and few signal might be amplified and few might be calibrated to lower level then the process is done as per the programmed by the user. Then the output data will be visible in the webpage as esp32 as a built in wifi module helps tp directly load the data in web with a good network connection and based on the data the alert signals will appear on the device screen.

3.3. COMPONENTS DESCRIPTIONS:

ESP32 DEV BOARD: The ESP32 is a powerful and affordable microcontroller widely used in electronics and IoT projects because it has built-in Wi-Fi and Bluetooth, a fast dual-core processor, and plenty of GPIO pins to connect sensors and modules. It supports features like ADC, DAC, PWM, I²C, SPI, and touch sensing, making it more advanced than the ESP8266.

DHT22 AND DS18B20: The DHT22 and DS18B20 are widely used digital temperature sensors, each designed for different needs. The DHT22 measures both temperature and humidity, making it useful for environment-monitoring applications like weather stations and health monitoring systems. The DS18B20, on the other hand, is a high-precision temperature-only sensor known for its accuracy, reliability, and ability to work with long cables. It supports the One-Wire protocol, allowing many sensors to connect on a single data pin, making it ideal for applications like body temperature measurement, industrial temperature sensing, and water or outdoor monitoring. While the DHT22 is best when you need both humidity and temperature, the DS18B20 is preferred when you need highly accurate, stable, and durable temperature readings.

MAX30100/102: The MAX30100 and MAX30102 are integrated optical sensors used for pulse rate and SpO₂ (blood oxygen level) measurement. Both sensors work using photoplethysmography (PPG), where infrared and red LEDs shine light into the skin, and a photodetector measures the light reflected back from blood flow. The MAX30102 is an improved and more stable version with better LED control, higher sensitivity, lower noise making it the preferred choice in most projects. These sensors interface easily with microcontrollers like the ESP32 via I²C and are commonly used in health monitoring systems, wearable devices, and fitness tracking applications.

MQ135: The MQ135 is a popular gas sensor used to measure air quality by detecting harmful gases present in the environment. It works by using a chemical-sensitive layer whose resistance changes when gases like ammonia (NH₃), nitrogen oxides (NO_x), benzene, smoke, CO₂, alcohol, and other pollutants are present in the air. The sensor outputs an analog voltage that can be read by a microcontroller such as the ESP32 to estimate air quality levels. It requires calibration for accurate results and becomes more stable after a few minutes of heating. MQ135 is commonly used in indoor air quality monitors, pollution detection systems, and health or environmental IoT projects.

CHAPTER-4

RESULTS AND DISCUSSIONS

This Chapter is the main core for the projects output status and it is the final part of the report which include the prototype of our project and how its performing and what are the results its displaying, its advantages, disadvantages and its future scope and references.

4.1. HARDWARE PROTOTYPE:

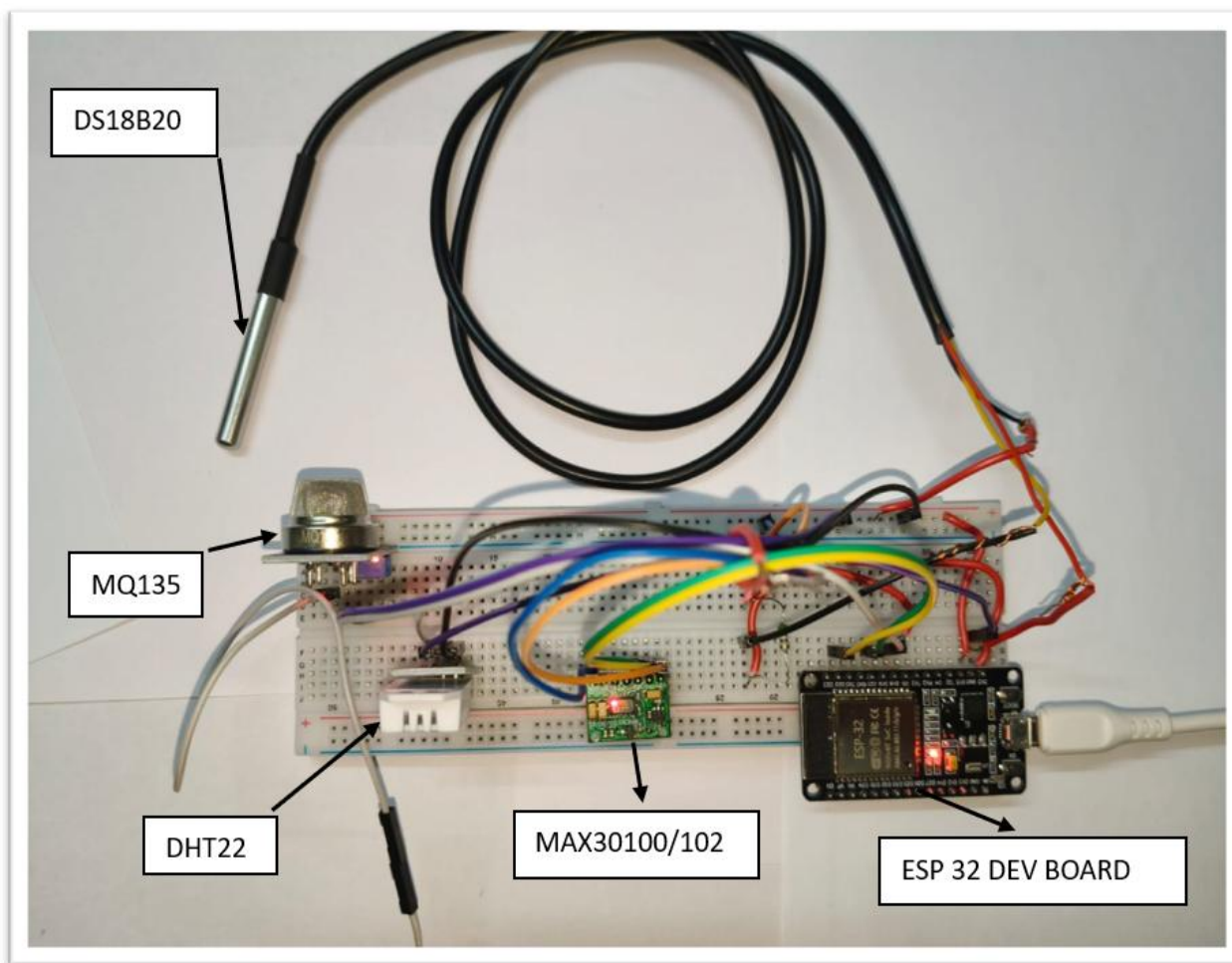


Fig.3. Prototype of the system

The prototype consists of four sensors that are DHT22, MAX30100/102, MQ135, DS18B20 and ESP32 board all these are assembled on a breadboard with help of jumper wires. The SCL and SDL of MAX30100 are connected to D22 and D20 pins of the ESP32. A0 point of MQ135 is connected to the D34, data pin of DHT22 is connected to D18 pin of ESP32 and Data wire of DS18B20 is connected to D5 of ESP32. Remaining VCC and GND pins are all connected to their respective vin and gnd pins as shown in the above image. Then the ESP32 is powered by external micro USB cable for stable power supply.

4.1.1. Results:

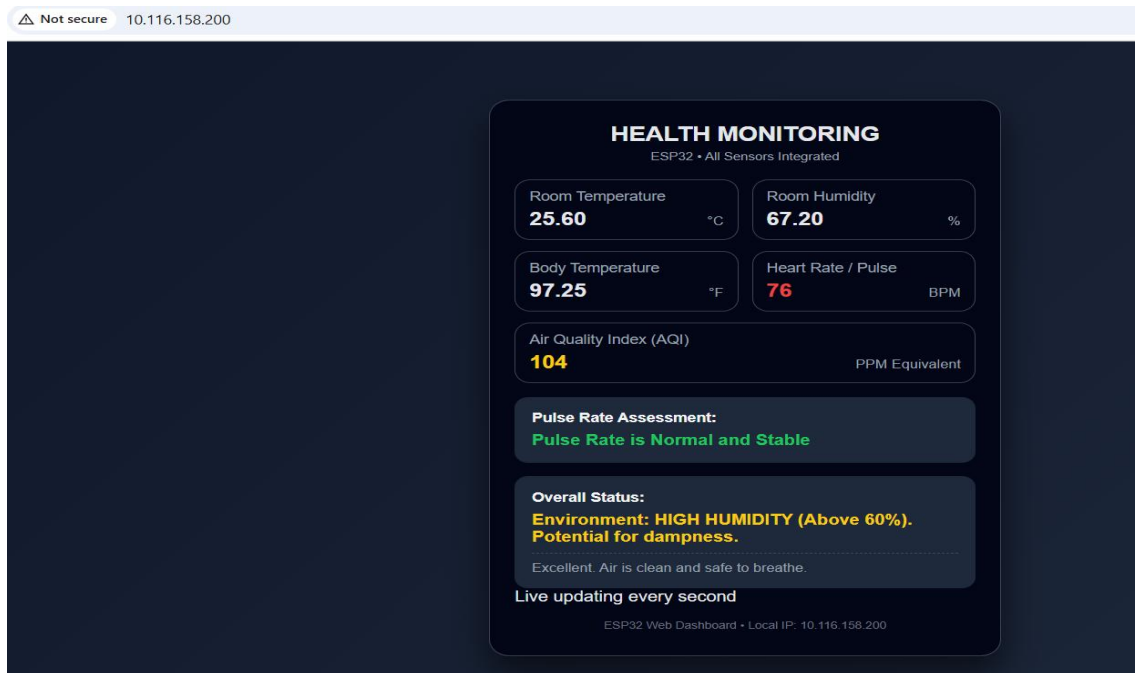


Fig.4. Webpage displaying output

The above figure shows the output data from the sensors which is displaying on the webpage. The data is kept on updating every second as the new readings are processed by the ESP32. It can be displayed on a webpage because of the WiFi module present in ESP32, which helps in displaying data on a webpage with a stable network connection. In the above figure, it shows the temperature of the room in Celsius and Humidity in percentage, Body temperature in Fahrenheit, Heart rate in bpm, Air quality index in ppm, and the complete data assessment with appropriate statements displayed.

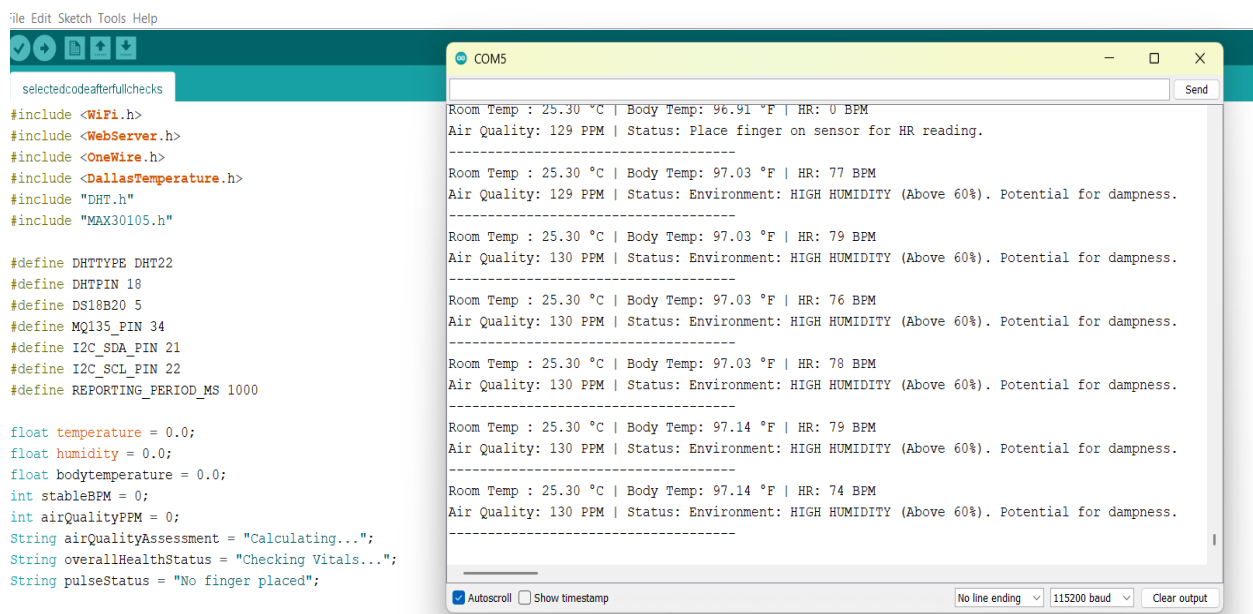


Fig.5. Serial output in Arduino IDE

The above figure Shows the readings of the sensors in the serial output of Arduino ide app, which the software used for implement the program in ESP32 board. It displays readings which are also visible in the webpage. This can be viewed as raw output for our webpage display.

4.1.2. Data Acquisition and Accuracy Verification:

- **Real-Time Data Collection:** Successful results would demonstrate the system's capability to accurately and continuously measure the defined vital signs (pulse and body temperature) of the user in real-time.
- **Signal Processing Validation:** Discussion would confirm that the Arduino program successfully integrated the sensors with the ESP32 and verified the data output. This includes successful implementation of the signal conditioning / analog front end and the microcontroller / mcu (adc + processing) blocks to ensure accurate pulse rate calculation from the raw PPG signal.
- **Performance Check:** A crucial discussion point would be the results of the final step: checking the performance and accuracy of the entire system against accepted standards.

4.1.3. Timely Alerting and Display Functionality:

- **Timely Alerting:** Results must show that the system can automatically detect abnormal health conditions (when a vital sign deviates from a safe range) and successfully activate the ALERT MODULE to send immediate warnings to the patient and/or caregivers.
- **Local Display:** Verification that the measured vital signs are reliably and continuously displayed on the on-device screen (OLED).

4.1.4. Remote Monitoring and IoT Functionality:

The most advanced results will pertain to the IoT integration:

- **IoT Connection Success:** Discussion would confirm the successful implementation of the IoT connection (Wi-Fi setup and cloud integration).
- **Remote Accessibility:** Results would verify that the data is successfully uploaded and monitored in real-time through the cloud platform, thereby fulfilling the objective to Enable Remote Monitoring for healthcare professionals, allowing intervention regardless of the patient's location.

4.1.5. Portability and Cost-Effectiveness:

- **Low-Cost Validation:** The discussion would confirm that the final system, utilizing affordable components like the ESP32 and basic sensors, serves as an affordable alternative to professional medical-grade devices.
- **Portability:** The successful integration of the POWER MANAGEMENT (BATTERY) block would confirm the system's intended portable nature.

4.2. ADVANTAGES OF THE SYSTEM:

The primary benefits are centered on improving accessibility, cost-efficiency, and the quality of health data collection.

- **Promotes Proactive Healthcare:** The system provides a proactive approach to personal healthcare by offering an early warning system for potential anomalies. This allows for timely intervention and prevents the escalation of a condition.
- **Cost-Effectiveness and Accessibility:** The project utilizes readily available and affordable components, serving as an affordable alternative to professional medical-grade devices. This makes proactive monitoring accessible and convenient for everyone.
- **Enhanced Convenience and Portability:** The portable design allows individuals to monitor their vital signs from the comfort of their home. This is particularly beneficial for Elderly individuals, Patients with chronic conditions, and Individuals in remote areas.
- **Data-Driven Empowerment:** The system enables real-time data collection, which can be stored, analyzed, and used by healthcare providers to identify trends and create personalized care plans. It also empowers individuals to take charge of their own health.
- **Remote Monitoring Capability:** Leveraging the Internet of Things (IoT) framework and Wireless Communication, the system provides valuable data for remote healthcare applications and allows healthcare professionals to track a patient's health status remotely.

4.3. DISADVANTAGES OF THE SYSTEM:

The disadvantages are primarily related to the constraints of using low-cost components for what is typically a professional-grade application.

- **Accuracy and Noise Susceptibility:** Low-cost biomedical sensors are generally more susceptible to noise (e.g., motion artifacts) than clinical-grade equipment. The need for a dedicated signal conditioning / analog front end highlights the challenge in ensuring reliable data acquisition.
- **Reliability and Durability:** Components like the breadboard and jumper wires used in the assembly phase suggest a prototype that may not offer the long-term reliability or ruggedness required for continuous, real-world, portable use compared to mass-produced consumer or medical devices.

4.4. APPLICATIONS:

Remote Patient Monitoring (RPM) and Home Care.

This is the most significant application, moving healthcare from the hospital to the patient's home.

- **Chronic Disease Management:** Continuous monitoring of vital signs (like heart rate) for conditions such as Congestive Heart Failure (CHF), COPD, and Hypertension. This allows doctors to adjust treatment plans remotely and identify issues before they become emergencies.
- **Elderly Care and Assisted Living:** Monitoring the health and safety of seniors, especially those living alone. Features often include:
- **Fall Detection:** Alerting caregivers or emergency services automatically if a fall is detected.
- **Activity Tracking:** Ensuring they maintain a healthy level of activity.
- **Post-Operative and Rehabilitation Care:** Monitoring a patient's recovery after surgery. This allows for early detection of complications, reduces the length of hospital stays, and lowers the cost of care.

Hospital and Clinical Settings: Even within a clinical environment, automated systems improve efficiency and care quality. **Continuous In-Hospital Monitoring:** Providing a real-time, less intrusive way to track patients' vitals in general wards, freeing up nursing staff from constant manual checks.

Personal Health and Wellness: To keep track on their health status and self evaluation.

Emergency Services and Alerting: **Timely Alerts:** Sending automated alerts to emergency contacts, doctors, or ambulances when a critical threshold is exceeded (e.g., heart rate drops too low). **Mobile Health (mHealth):** Providing real-time data to first responders or ambulance staff so they can be better prepared before the patient arrives at the hospital.

Industrial Labor Safety & Health Monitoring: **Physiological Strain and Fatigue Detection**
Heat Stress Prevention: Monitoring the Body Temperature and Heart Rate of workers in hot environments (e.g., near furnaces, boilers, or in non-air-conditioned spaces). If a worker's temperature or heart rate exceeds safe thresholds, an alert is triggered, preventing heat stroke or exhaustion.

4.5. FUTURE SCOPE OF WORK:

The current system establishes a foundation for low-cost, portable, and remote health monitoring. The future scope aims to significantly increase the system's capabilities, shifting it from a prototype to a more comprehensive and robust solution.

4.5.1. Expanded Monitoring and Sensor Integration

- **Integrate Additional Vital Signs:** Expand the system's functionality to include other critical parameters such as Blood Oxygen Saturation (SpO₂), Blood Pressure, and Electrocardiogram (ECG). This would require integrating sensors like the MAX30102 (which measures SpO₂ along with pulse) and specialized pressure/ECG modules.
- **Environmental Sensing for Context:** Utilize the existing capability to measure humidity and add sensors for air quality or ambient temperature to correlate external factors with patient vital signs, providing deeper contextual data.

- **Wearable Design:** Develop the final product into a more discreet, durable, and wearable form factor (e.g., a wristband or patch), moving beyond the breadboard assembly. This would necessitate custom Printed Circuit Board (PCB) design and integration of low-power flexible electronics.

4.5.2. Advanced Data Analysis and Intelligence

- **Machine Learning (ML) for Anomaly Detection:** Implement ML algorithms on the collected vital sign data to move beyond simple threshold-based alerting. This would allow the system to learn the user's personal baseline and detect subtle, complex anomalies indicative of conditions like early-stage sepsis or sleep apnea.
- **Predictive Analytics:** Use historical data stored on the central cloud platform to develop predictive models that forecast the probability of a health event (e.g., a hypertensive crisis) within a specified timeframe, enabling true preventative intervention.
- **Visualization and User Experience (UX):** Develop a dedicated, intuitive mobile application to complement the web dashboard, offering users and caregivers personalized trends, health reports, and simplified controls for the remote healthcare applications.

REFERENCE:

- [1]. Al-Fuqaha, Ala, et al. "Internet of things: A survey on enabling technologies, protocols, and applications." *IEEE communications surveys & tutorials* 17.4 (2015): 2347-2376.
- [2] Li, Chao, Xiangpei Hu, and Lili Zhang. "The IoT-based heart disease monitoring system for pervasive healthcare service." *Procedia computer science* 112 (2017): 2328-2334.
- [3] Lavanya, S., G. Lavanya, and J. Divyabharathi. "Remote prescription and I-Home healthcare based on IoT." *2017 International Conference on Innovations in Green Energy and Healthcare Technologies (IGEHT)*. IEEE, 2017.
- [4] Medvediev, Ivan, et al. "IoT solutions for health monitoring: analysis and case study." *2018 IEEE 9th International Conference on Dependable Systems, Services and Technologies (DESSERT)*. IEEE, 2018.
- [5] Pardeshi, Vivek, et al. "Health monitoring systems using IoT and Raspberry Pi—a review." *2017 International Conference on Innovative Mechanisms for Industry Applications (ICIMIA)*. IEEE, 2017.
- [6] Marhoon, Ali Fadel, and Ali Hussein Hamad. "A new real-time resource-efficient algorithm for ECG denoising, feature extraction and classification-based wearable sensor network." *International Journal of Biomedical Engineering and Technology* 18.2 (2015): 103-114.
- [7] Sehrash Ashraf, Shahnaz Parveen Khattak, Mohammad Tariq Iqbal, Design and implementation of an open-source and Internet-of-Things-based health monitoring system *Journal of Low Power Electronics and Applications* 13 (4), 57, 2023.
- [8] P. Anirudh, G. A. E. S. Kumar, R. P. Vidyadhar, G. Pranav and B. A. Aumar, "Automatic Patient Monitoring and Alerting System based on IoT", *2023 8th International Conference on Communication and Electronics Systems (ICCES)*, Coimbatore, India, 2023, pp. 328-331.
- [9] Stradolini, Francesca, et al. "Live demonstration: An IoT smartwatch-based system for intensive care monitoring." *2017 IEEE Biomedical Circuits and Systems Conference (BioCAS)*. IEEE, 2017.
- [10] Zilani, Kazi Abu, et al. "R 3 HMS, An IoT Based Approach for Patient Health Monitoring." *2018 International Conference on Computer, Communication, Chemical, Material and Electronic Engineering (IC4ME2)*. IEEE, 2018.
- [11] Yi, Ding, et al. "Design and implementation of mobile health monitoring protocol." *2016 system IEEE based Advanced on MQTT Information Management, Communicates, Electronic and Automation Control Conference (IMCEC)*. IEEE, 2016.
- [12] Gardašević, Gordana, et al. "A heterogeneous IoT-based architecture for remote monitoring of physiological and environmental parameters." *International Conference on IoT Technologies for HealthCare*. Springer, Cham, 2017.
- [13] Lekić, Milica, and Gordana Gardašević. "IoT sensor integration to International Node-RED platform." *2018 17th Symposium (INFOTEH)*. IEEE, 2018.

- [14] M. K. K. P. Devi, P. K. Devi, and K. V. L. N. Devi, "A Portable IoT-Based Health Monitoring Framework Using ESP32 for Isolated and Home-Based Patient Care," *REST Publisher*, vol. 5, no. 5, pp. 1–6, May 2025.
- [15] R. E. L. A. S. R. B. H. M. P. G. V. Subbaiah, "Real-Time Remote Health and Safety Monitoring using MAX30100 and MQ-135 with ESP32," *IEEE J. Biomed. Health Inform.*, vol. 28, no. 1, pp. 195–204, Jan. 2024.
- [33] P. S. B. Macheso and A. G. Meela, "IoT Based Patient Health Monitoring using ESP8266 and Arduino," *Int. J. Comput. Commun. Inf.*, vol. 7, pp. 75–83, Mar. 2022.

APPENDIX: CODE FOR ESP32 AND ITS WEBPAGE

```
#include <WiFi.h>

#include <WebServer.h>

#include <OneWire.h>

#include <DallasTemperature.h>

#include "DHT.h"

#include "MAX30105.h"

#define DHTTYPE DHT22

#define DHTPIN 18

#define DS18B20 5

#define MQ135_PIN 34

#define I2C_SDA_PIN 21

#define I2C_SCL_PIN 22

#define REPORTING_PERIOD_MS 1000

float temperature = 0.0;

float humidity = 0.0;

float bodytemperature = 0.0;

int stableBPM = 0;

int airQualityPPM = 0;

String airQualityAssessment = "Calculating...";

String overallHealthStatus = "Checking Vitals...";

String pulseStatus = "No finger placed";

#define ROOM_TEMP_COLD_LIMIT 18.0

#define ROOM_TEMP_HOT_LIMIT 26.0

#define NORMAL_HUMIDITY_HIGH 60.0

#define NORMAL_BPM_LOW 60

#define NORMAL_BPM_HIGH 100

#define AQI_GOOD_LIMIT 600

#define AQI_MIN_FLOOR 5

const char* ssid = "OnePlus Nord CE";

const char* password = "12042005";

IPAddress local_IP(10,116,158,200);
```

```
IPAddress gateway(10,116,158,144);
IPAddress subnet(255,255,255,0);
IPAddress primaryDNS(8,8,8,8);
IPAddress secondaryDNS(8,8,4,4);
DHT dht(DHTPIN, DHTTYPE);
OneWire oneWire(DS18B20);
DallasTemperature sensors(&oneWire);
WebServer server(80);
MAX30105 particleSensor;
uint32_t tsLastReport = 0;
const int FILTER_WINDOW = 10;
long irSum = 0;
long irReadings[FILTER_WINDOW];
int readingIndex = 0;
bool readingsReady = false;
const int BPM_AVERAGE_SIZE = 8;
int bpmHistory[BPM_AVERAGE_SIZE] = {0};
int bpmIndex = 0;
int currentBPM = 0;
long lastBeatTime = 0;
int irAC = 0;
int minAC = 100000;
int maxAC = 0;
bool beatDetected = false;
int threshold = 3500;
void handleRoot();
void handleData();
void handle_NotFound();
String buildPage();
long getSimpleRunningAverage(long newReading);
int getBPMFinalAverage();
int readRawAirQuality();
String getAirQualityAssessment(int ppm);
```

```
String getHealthStatus();

String getPulseHealthStatus();

long getSimpleRunningAverage(long newReading) {
    irSum = irSum - irReadings[readingIndex];
    irReadings[readingIndex] = newReading;
    irSum = irSum + newReading;
    readingIndex++;
    if (readingIndex >= FILTER_WINDOW) {
        readingIndex = 0;
        readingsReady = true;}
    return (readingsReady) ? (irSum / FILTER_WINDOW) : newReading;}

int getBPMFinalAverage() {
    long sum = 0;
    for (int i = 0; i < BPM_AVERAGE_SIZE; i++) {
        sum += bpmHistory[i];}
    return sum / BPM_AVERAGE_SIZE;}

int readRawAirQuality() {
    return analogRead(MQ135_PIN);}

String getAirQualityAssessment(int ppm) {
    if (ppm < 350) return "Excellent. Air is clean and safe to breathe.";
    if (ppm < 600) return "Good. Minor pollutants detected.";
    if (ppm < 850) return "Moderate. Ventilation recommended.";
    return "Poor. Air quality is poor and may affect health. Take action.";}

String getPulseHealthStatus() {
    if (stableBPM == 0) {
        return "No finger placed";}
    if (stableBPM >= NORMAL_BPM_LOW && stableBPM <= NORMAL_BPM_HIGH) {
        return "Pulse Rate is Normal and Stable";}
    if (stableBPM < NORMAL_BPM_LOW) {
        return "Pulse Rate is Low (Bradycardia Risk);}
    if (stableBPM > NORMAL_BPM_HIGH) {
        return "Pulse Rate is High (Tachycardia Risk);}
    return "Pulse Reading Unstable";}
```



```
String getHealthStatus() {  
    if (bodytemperature > 38.0) {  
        return "Patient Vitals: Elevated Body Temperature (Fever Risk).";  
    }  
    if (stableBPM == 0) {  
        return "Place finger on sensor for HR reading.";  
    }  
    if (stableBPM > NORMAL_BPM_HIGH) {  
        return "Patient Vitals: High Heart Rate detected.";  
    }  
    if (stableBPM < NORMAL_BPM_LOW && stableBPM > 0) {  
        return "Patient Vitals: Low Heart Rate detected.";  
    }  
    if (temperature > ROOM_TEMP_HOT_LIMIT) {  
        return "Environment: ROOM IS HOT (Above 26°C). Recommend cooling.";  
    }  
    if (temperature < ROOM_TEMP_COLD_LIMIT && temperature > 0) {  
        return "Environment: ROOM IS COLD (Below 18°C). Recommend heating.";  
    }  
    if (humidity > NORMAL_HUMIDITY_HIGH) {  
        return "Environment: HIGH HUMIDITY (Above 60%). Potential for dampness.";  
    }  
    if (airQualityPPM > AQI_GOOD_LIMIT) {  
        return "Environment: POOR AIR QUALITY. Increase ventilation immediately.";  
    }  
    if (bodytemperature > 0 && stableBPM > 0 && airQualityPPM <= AQI_GOOD_LIMIT) {  
        return "ALL VITAL SIGNS ARE STABLE. Environment is good.";  
    }  
    return "Checking Vitals...";  
}  
  
void setup() {  
    Serial.begin(115200);  
    randomSeed(analogRead(0));  
    delay(200);  
    WiFi.config(local_IP, gateway, subnet, primaryDNS, secondaryDNS);  
    WiFi.begin(ssid, password);  
    while (WiFi.status() != WL_CONNECTED) {  
        Serial.print(".");  
        delay(700);  
        Serial.println();  
        Serial.println("WiFi connected");  
        Serial.print("ESP32 Static IP: ");  
        Serial.println(WiFi.localIP());  
    }
```

```
dht.begin();

sensors.begin();

Wire.begin(I2C_SDA_PIN, I2C_SCL_PIN);

if (!particleSensor.begin(Wire)) {

Serial.println("MAX30102 Initialization FAILED! Check I2C.");}

else {

particleSensor.setup(60, 4, 2, 100, 411, 4096);

particleSensor.clearFIFO();

Serial.println("MAX30102 initialized successfully.");}

pinMode(MQ135_PIN, INPUT);

for (int i = 0; i < FILTER_WINDOW; i++) {

irReadings[i] = 0;}

airQualityPPM = 0;

server.on("/", handleRoot);

server.on("/data", handleData);

server.onNotFound(handle_NotFound);

server.begin();

Serial.println("Web server started");}

void loop() {

server.handleClient();

sensors.requestTemperatures();

temperature    = dht.readTemperature();

humidity       = dht.readHumidity();

bodytemperature = sensors.getTempCByIndex(0);

airQualityPPM = readRawAirQuality();

particleSensor.check();

long irValue = particleSensor.getIR();

long irAvg = getSimpleRunningAverage(irValue);

if (irValue > 50000) {

irAC = irValue - irAvg;

if (irAC > maxAC) maxAC = irAC;

if (irAC < minAC) minAC = irAC;

if (irAC < (maxAC - threshold) && beatDetected == true) {
```

```
beatDetected = false;

long timeDifference = millis() - lastBeatTime;

lastBeatTime = millis();

if (timeDifference > 300 && timeDifference < 2000) {
  currentBPM = 60000 / timeDifference;}

maxAC = irAC;

minAC = 100000;} else if (irAC > (minAC + threshold)) {

beatDetected = true;

maxAC = irAC;}

if (currentBPM > 40 && currentBPM < 200) {

stableBPM = currentBPM;}

} else {

stableBPM = 0;

currentBPM = 0;

maxAC = 0;

minAC = 100000;

beatDetected = false;}

airQualityAssessment = getAirQualityAssessment(airQualityPPM);

overallHealthStatus = getHealthStatus();

pulseStatus = getPulseHealthStatus();

if (millis() - tsLastReport > REPORTING_PERIOD_MS) {

Serial.printf("Room Temp : %.2f °C | Body Temp: %.2f °C | HR: %d BPM\n", temperature, bodytemperature,
stableBPM);

Serial.printf("Air Quality RAW ADC: %d | Status: %s\n", airQualityPPM, overallHealthStatus.c_str());

Serial.println("-----");

tsLastReport = millis();}}

void handleRoot() {

server.send(200, "text/html", buildPage());

}

void handleData() {

String json = "{";

json += "\"temp\":" + String(temperature, 2) + ",";

json += "\"hum\":" + String(humidity, 2) + ",";
```

```
json += "\"body\": \" " + String(bodytemperature, 2) + " \";  
json += "\"hr\": \" " + String(stableBPM) + " \";  
json += "\"air\": \" " + String(airQualityPPM) + " \";  
json += "\"status\": \" " + overallHealthStatus + " \";  
json += "\"air_status\": \" " + airQualityAssessment + " \";  
json += "\"pulse_status\": \" " + pulseStatus + " \";  
json += "}\"";  
server.send(200, "application/json", json);}   
  
void handle_NotFound() {  
server.send(404, "text/plain", "404: Not found");}  
  
String buildPage() {  
String page = R"rawliteral(  
<!DOCTYPE html>  
  
<html>  
  
<head>  
  
<meta charset="UTF-8">  
  
<title>ESP32 HEALTH MONITORING</title>  
  
<meta name="viewport" content="width=device-width, initial-scale=1">  
  
<style>  
  
*{box-sizing:border-box;margin:0;padding:0;}  
  
body{  
  
font-family: Arial, Helvetica, sans-serif;  
  
background: linear-gradient(135deg,#0f172a,#1e293b);  
  
color:#e5e7eb;  
  
min-height:100vh;  
  
display:flex;  
  
align-items:center;  
  
justify-content:center;  
  
padding:16px;}  
  
.card{  
  
background:#020617;  
  
border-radius:18px;  
  
padding:24px 22px 26px;
```

```
max-width:450px;

width:100%;

box-shadow:0 18px 35px rgba(0,0,0,0.6);

border:1px solid rgba(148,163,184,0.4);

}

.title{ font-size:22px; font-weight:bold; margin-bottom:4px; text-align:center;}

.subtitle{ font-size:12px; color:#9ca3af; text-align:center; margin-bottom:18px;}

.grid{ display:grid; grid-template-columns:1fr 1fr; gap:12px; }

.metric{

background:#020617;

border-radius:14px;

padding:10px 12px;

border:1px solid rgba(75,85,99,0.8);

display:flex;

flex-direction:column;}

.label{ font-size:14px; color:#9ca3af; margin-bottom: 4px; }

.value-unit-group{ display: flex; justify-content: space-between; align-items: baseline; }

.value{ font-size:20px; font-weight:bold; color:#e5e7eb; }

.unit{ font-size:13px; color:#9ca3af; margin-left:3px;}

.assessment-box, .pulse-status-box {

margin-top:15px;

padding:15px;

border-radius:12px;

background:#1e293b;}

.assessment-title, .pulse-title {

font-weight:bold;

font-size:14px;

color:#f3f4f6;

margin-bottom:6px;}

.assessment-status, .pulse-status-text {

font-size:16px;

font-weight:600;}

.air-status-line {
```

```
font-size: 13px;
color: #9ca3af;
margin-top: 8px;
border-top: 1px dashed rgba(148,163,184,0.3);
padding-top: 8px;}
.status-alert, .pulse-abnormal { color: #dc2626 !important; }
.status-warning { color: #facc15 !important; }
.status-stable, .pulse-normal { color: #22c55e !important; }
.pulse-no-finger { color: #9ca3af !important; }
.heart-rate-value { color: #ef4444; }
.air-quality-value { color: #facc15; }
.footer{ margin-top:16px; font-size:11px; text-align:center; color:#6b7280; }
</style>
<script>
function updateData(){
fetch('/data')
.then(res => res.json())
then(d => {
if (d.pulse_status.includes("No finger")) {
pulseElement.classList.add('pulse-no-finger');
} else if (d.pulse_status.includes("Normal")) {
pulseElement.classList.add('pulse-normal');
} else {
pulseElement.classList.add('pulse-abnormal');}
const statusElement = document.getElementById('status');
statusElement.classList.remove('status-alert', 'status-warning', 'status-stable');
if (d.status.includes("ALERT")) {
statusElement.classList.add('status-alert');
} else if (d.status.includes("Vitals") || d.status.includes("Environment") || d.status.includes("POOR AIR")) {
statusElement.classList.add('status-warning');
} else if (d.status.includes("ALL VITAL SIGNS")) {
statusElement.classList.add('status-stable');}}}
.catch(err => console.log(err));}
```

```
setInterval(updateData, 1000);

window.onload = updateData;

</script>

</head>

<body>

<div class="card">

<div class="title">HEALTH MONITORING</div>

<div class="subtitle">ESP32 • All Sensors Integrated</div>

<div class="grid">

<div class="metric">

<div class="label">Room Temperature</div>

<div class="value-unit-group">

<span class="value" id="temp">--</span>

<span class="unit">&deg;C</span>

</div>

</div>

<div class="metric">

<div class="label">Room Humidity</div>

<div class="value-unit-group">

<span class="value" id="hum">--</span>

<span class="unit">%</span>

</div>

</div>

<div class="metric">

<div class="label">Body Temperature</div>

<div class="value-unit-group">

<span class="value" id="body">--</span>

<span class="unit">&deg;C</span>

</div>

</div>

<div class="metric">

<div class="label">Heart Rate / Pulse</div>

<div class="value-unit-group">
```

```
<span class="value heart-rate-value" id="hr">--</span>
<span class="unit">BPM</span>
</div>
</div>
<div class="metric" style="grid-column: 1 / span 2;">
<div class="label">Air Quality (RAW ADC)</div>
<div class="value-unit-group">
<span class="value air-quality-value" id="air">--</span>
<span class="unit">ADC 0-4095</span>
</div>
</div>
</div>
<div class="pulse-status-box" style="grid-column: 1 / span 2;">
<div class="pulse-title">Pulse Rate Assessment:</div>
<div class="pulse-status-text" id="pulse_status_text">Checking Pulse...</div>
</div>
<div class="assessment-box">
<div class="assessment-title">Overall Status:</div>
<div class="assessment-status" id="status">Checking Vitals...</div>
<div class="air-status-line" id="air_status_line"></div>
</div>
<div class="badge">Live updating every second</div>
<div class="footer">
ESP32 Web Dashboard • Local IP: )rawliteral";
page += WiFi.localIP().toString();
page += R"rawliteral(
</div>
</div>
</body>
</html>)<rawliteral";return page;}
```